IMPLEMENTATION GUIDE

Real Estate Lead Automation on n8n

From an ad click to a booked site visit, entirely over WhatsApp — with Google Sheets as the CRM, Claude grading every lead, and a live dashboard over the whole thing. Everything runs on your own machine.

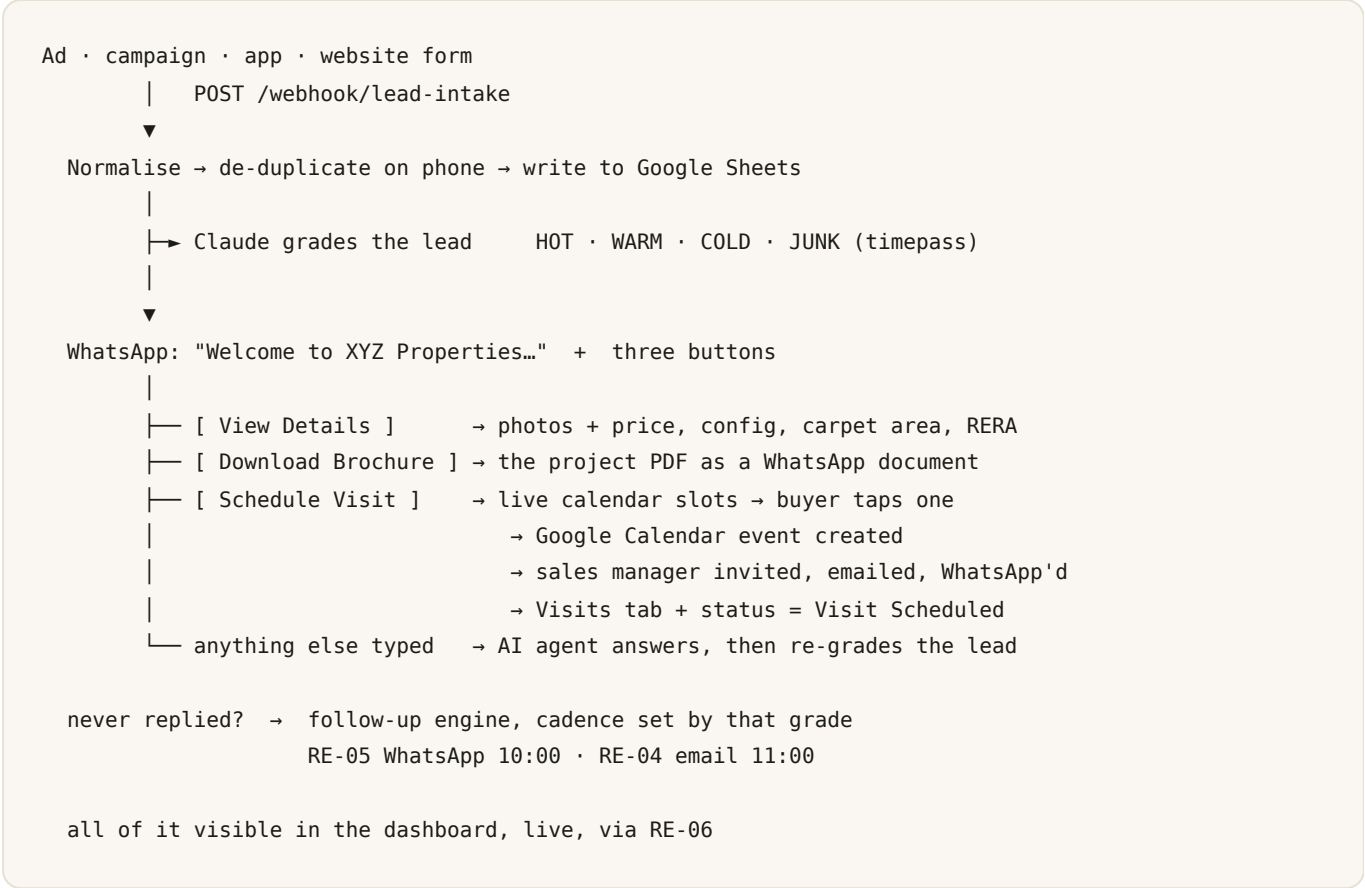
4 workflows
RE-00, RE-04, RE-05, RE-06 plus the CRM workbook and dashboard

~90 minutes
first-time setup excluding Meta template review

Runs on
localhost:5678 tunnel needed for inbound

What you are building

Read this page before touching anything — it is the whole system in one picture.



The five files you import

FILE	WHAT IT DOES
`RE-00`	Lead capture, the three-button WhatsApp menu, details, brochure, site-visit booking, the AI agent for free text, and AI lead grading. The main workflow.
`RE-05`	WhatsApp follow-up engine. Daily sweep at 10:00 IST.
`RE-04`	Email follow-up on the same cadence at 11:00 IST, plus the manager's morning pipeline digest at 08:30.
`RE-06`	Read-only JSON feed that powers the dashboard.
`XYZ-Properties-CRM.xlsx`	The Google Sheets workbook: Leads, Projects, Visits, Messages, plus a dashboard and field guide.

Order of operations

01 Install and run n8n 15 min

02	Open a tunnel so Meta can reach you	5 min · required for inbound
03	Create the CRM spreadsheet	10 min
04	Create the five credentials in n8n	20 min
05	Import the workflows and fill in Config	10 min
06	Submit the two WhatsApp templates to Meta	10 min + review wait
07	Point Meta's webhook at your tunnel	5 min
08	Test the whole flow locally	15 min
09	Connect the dashboard	5 min
10	Go live	checklist on the last page

> **The one thing that trips everyone up**
>
> A buyer who filled in a form has **not** messaged you, so Meta's 24-hour customer-service window is closed and your first message **must** be a pre-approved template. Submit both templates (step 6) early — review can take anywhere from minutes to a day. Everything after the buyer's first tap is free-form and needs no approval.

Steps 1 – 3 · Machine, tunnel, CRM

Get n8n running, make it reachable from the internet, and build the spreadsheet everything writes to.

1 Install and run n8n on localhost

Either works. Docker is easier to keep clean; npx is faster to try.

```
# Docker — set WEBHOOK_URL to your tunnel host from step 2
docker run -it --rm -p 5678:5678 \
  -e WEBHOOK_URL="https://your-subdomain.ngrok-free.app/" \
  -e GENERIC_TIMEZONE="Asia/Kolkata" \
  -v n8n_data:/home/node/.n8n n8nio/n8n

# or, with Node 20+ installed
npx n8n
```

Open `http://localhost:5678` and create the owner account. Leave it running.

> **Why `WEBHOOK_URL` matters.** Without it, n8n prints webhook URLs starting `http://localhost:5678`. Meta cannot call those. Set it to the tunnel host and the URLs n8n shows you are the ones you can actually paste into Meta.

2 Open a tunnel

Meta's servers must reach your machine over public HTTPS. Localhost is not reachable, and there is no way around it.

```
ngrok http 5678       # or: cloudflared tunnel --url http://localhost:5678
```

Copy the `https://….ngrok-free.app` address. That host replaces `localhost:5678` in every URL you give Meta.

> **The free ngrok URL changes every restart.** When it does, update `WEBHOOK_URL`, restart n8n, and re-save the callback URL in Meta. A reserved domain or cloudflared removes this chore — worth it before a client demo.

If you are already running behind ngrok

POINT	WHAT TO DO
Publishing is unaffected	Activation happens inside n8n. A tunnel is irrelevant to it — a workflow that will not publish has a node issue, not a network problem.
`WEBHOOK_URL` needs a restart	n8n reads it at boot. Setting it while n8n is running changes nothing; the node still shows a `localhost` URL. Restart, then re-copy the Production URL.
Check it took effect	Open any webhook node. Its Production URL should start with your ngrok host, not `localhost:5678`. If it does not, `WEBHOOK_URL` did not apply.
Point the dashboard at localhost	Use `http://localhost:5678/webhook/dashboard-data`, not the ngrok URL. It is the same machine, so it is faster, uses no tunnel bandwidth, and sidesteps ngrok's browser interstitial, which can return an HTML warning page where the dashboard expects JSON.
Test through the tunnel with curl	`curl https://your-host.ngrok-free.app/webhook/lead-intake` — curl is not treated as a browser, so no interstitial. If that reaches n8n, Meta will too.
Keep the tunnel on 5678	`ngrok http 5678`. Tunnelling any other port silently produces a URL that answers nothing.

3 Create the CRM spreadsheet

Everything lives in one workbook, `XYZ-Properties-CRM.xlsx`.

1. Put the file in Google Drive → right-click → **Open with → Google Sheets → File → Save as Google Sheets**.
2. Copy the ID out of the URL: `docs.google.com/spreadsheets/d/` **THIS-PART** `/edit#gid=0`. This is the **whole file's** ID, not a tab's. Ignore the `gid=` at the end — that identifies a single tab and is never used. Each node picks its tab by *name*, which is why renaming a tab breaks things.
3. Open the **Projects** tab and replace the two shaded example rows with your real projects.
4. Share the sheet with the Google account n8n will authenticate as, with **Editor** access.

TAB	WHO WRITES IT	NOTES
Leads	n8n	One row per buyer, matched on `phone`. Yours to edit: `owner`, `notes`.
Projects	**You**	The only tab you fill in. Every word the bot says about a property is read from here.
Visits	n8n	One row per booked site visit.
Messages	n8n	Full inbound/outbound log.
Dashboard · Field guide	—	Live formulas, and what every column means.

> **Two rules that silently break everything**
>
> **Never rename or reorder a column, and keep row 1 as the header.** The workflows map to headers by name — a renamed column just stops being written, with no error.
>
> **Image and brochure links must be public direct links.** Meta fetches them server-side. A Google Drive "share" link returns an HTML page, not a file, and the send fails.

Steps 4 – 5 · Credentials and workflows

Five credentials, four imports, one Config node each.

4

Create the credentials

In n8n: **Credentials → Add credential**. Create all five before importing, so the imported nodes can find them.

CREDENTIAL	WHERE THE VALUES COME FROM
WhatsApp API	Meta → your app → WhatsApp → API Setup. You need the permanent access token and the Business Account ID. Used by every outbound message.
WhatsApp OAuth / Trigger	App ID and App Secret from the same Meta app. Used only by the inbound trigger, which also handles Meta's verification handshake.
Google Sheets OAuth2	Google Cloud console → OAuth client. Sign in as the account the sheet is shared with.
Google Calendar OAuth2	Same project, Calendar API enabled.
Anthropic	An API key from the Claude console. Powers grading, the free-text agent and follow-up copy.

> **No secret is stored inside the workflow files.** Every WhatsApp and Claude HTTP node uses n8n's *Predefined Credential Type*, so tokens live in n8n's credential store and the JSON stays safe to email, commit or hand to a client.

5 Import the workflows and fill in Config

Workflows → ⋯ → Import from File, once per file. Then open the **Config** node in each and set the values. Config is the only node you need to edit.

FIELD	EXAMPLE	IN
`COMPANY_NAME`	XYZ Properties	all
`SHEET_ID`	the ID from step 3 — the whole spreadsheet file, not a tab	all
`WA_PHONE_NUMBER_ID`	Meta → WhatsApp → API Setup	00, 05, 06
`CALENDAR_ID`	`primary` or the site-visit calendar	00
`SALES_MANAGER_EMAIL` / `_WHATSAPP`	who receives the visit	all
`USE_APPROVED_TEMPLATE`	`true` live, `false` only for testing	00
`CLAUDE_MODEL`	`claude-opus-5`	00, 04, 05
`BOOKING_START_HOUR` / `_END_HOUR`	`10` / `18` — site office hours	00
`VISIT_DURATION_MIN`	`45`	00
Cadence and `MAX_FOLLOWUPS`	`3 / 5 / 21` and `6`	04, 05, 06
`DASHBOARD_KEY`	any string you choose	06

> **Keep the cadence numbers identical in RE-04, RE-05 and RE-06.** The two follow-up engines avoid double-messaging by sharing `last_touch` and running an hour apart. If their numbers drift, that protection stops working and buyers get two messages the same morning. RE-06 uses the same numbers to predict the queue — if it disagrees, the dashboard lies.

Finally, switch each workflow to **Active**.

Steps 6 – 7 · WhatsApp templates and the webhook

The only part that depends on someone else's approval queue. Do it early.

6 Submit the two templates

Meta Business Manager → **WhatsApp Manager → Message Templates → Create**.

Template A — the welcome menu

Name	`lead_welcome_menu`
Category	Marketing (or Utility)
Language	`en`
Body	Hi {{1}}, welcome to {{2}}! Thanks for your interest in {{3}}. I can share the full details and photos, send you the brochure, or book a site visit at a time that suits you.
Buttons	Quick Reply × 3, **in this exact order**: `View Details` · `Download Brochure` · `Schedule Visit`

Template B — the follow-up

Name	`followup_update`
Category	Marketing
Body	Hi {{1}}, an update on {{2}}: {{3}}
Buttons	Quick Reply × 3, same three, same order

> **Button order is not cosmetic.** The workflows send payloads against button *index* 0, 1 and 2 — `VIEW_DETAILS`, `DOWNLOAD_BROCHURE`, `BOOK_VISIT`. Reorder the buttons in Meta and taps will trigger the wrong branch.

> Claude writes only `{{3}}` in the follow-up. The template text itself never changes, so it stays approved forever while every message still says something new.

Starting on a Meta test number, before any template exists

This is where every implementation begins, and templates are not needed for it. A test number can send free-form messages — including the three-button menu — to anyone who has messaged you in the last 24 hours.

#	STEP
1	Meta → **WhatsApp → API Setup** → the **To** field → *Manage phone number list*. Add your own number and confirm the code. A test number can only message numbers on this list (max 5); anything else fails with `131030`.
2	From your phone, WhatsApp any message to the test number in the **From** dropdown. That opens the 24-hour window.
3	RE-00 → **Config** → set `USE_APPROVED_TEMPLATE` to `false`. Save, publish.
4	Submit the lead form. The welcome arrives with all three buttons, no approval involved.
5	Drive the button branches through `/webhook/simulate` until the inbound webhook is wired.

> **The test-number token expires after 24 hours**
>
> The token on the API Setup page is temporary. Tomorrow every send fails with error `190` and it will look like a fresh bug. For anything longer than a day, create a System User in Business Settings and generate a permanent token.

> **Leave RE-04 and RE-05 unpublished until templates exist.** Their follow-ups are template-only by design — a lead who has not replied is outside the window — so every send would fail. Nothing breaks if they do run, since the touch counter only advances when Meta accepts the message, but there is no point paying for the model calls.

7

Point Meta at your inbound webhook

1. In n8n, open RE-00 → the **WhatsApp Message Received** node → copy its **Production URL**.
2. Replace `localhost:5678` in that URL with your tunnel host.
3. Meta app → **WhatsApp → Configuration → Callback URL** = that URL. Verify token = the one in your n8n WhatsApp Trigger credential.
4. Press **Verify and save**, then **subscribe to the `messages` field**. Nothing arrives without that subscription.

Verification fails? Nine times out of ten the workflow is not published, the trigger node is disabled (a disabled node registers no webhook, so the URL genuinely does not exist), the tunnel is down, or the verify token does not match. Meta's error is the same for all four. Reproduce Meta's check yourself — it is a plain GET:

```
curl -i "<your callback url>?hub.mode=subscribe&hub.challenge=12345&hub.verify_token=
<your token>"
```

`12345` back means n8n is fine. A 404 means the trigger is not registered.

Verified but nothing ever arrives: check which app the WABA is subscribed to

This one costs people hours. Ticking `messages` configures your *app's* webhook, but events only flow if the **WhatsApp Business Account** is separately subscribed to that app. Out of the box a test WABA is often subscribed to Meta's own `WA DevX Webhook Events 1P App` instead — the one behind the test panel in the API Setup page — so everything verifies and nothing is delivered.

```
curl "https://graph.facebook.com/v21.0/<WABA_ID>/subscribed_apps?access_token=
<TOKEN>"
```

If your app is not in that list, add it. The POST subscribes whichever app owns the token:

```
curl -X POST "https://graph.facebook.com/v21.0/<WABA_ID>/subscribed_apps?
access_token=<TOKEN>"
```

It is the **WhatsApp Business Account ID**, not the Phone number ID — they sit next to each other and look alike. Using the wrong one gives *"Tried accessing nonexisting field (subscribed_apps)"*. To read the WABA id straight off your token: `GET /v21.0/debug_token?input_token=TOKEN&access_token=TOKEN` → `granular_scopes` → `target_ids`.

The fastest way to see what Meta is actually doing is ngrok's request inspector at `http://127.0.0.1:4040`. Filter it to `whatsapp` and tap a button: no request means Meta is not sending, a 401 or 403 means n8n rejected the signature (wrong App Secret), and a 200 means the event landed and the execution is in n8n's **Executions** tab. Browse n8n at `localhost:5678` rather than through the tunnel, or its own UI traffic buries everything.

Steps 8 – 9 · Test it, then watch it

Prove each stage works on your own number before any real lead arrives.

8

Test the whole flow locally

Open `test/lead-form.html` in a browser, or drive it from the terminal:

```
# 1 · a lead from a campaign — writes the CRM, grades it, sends the menu
curl -X POST http://localhost:5678/webhook/lead-intake \
  -H 'content-type: application/json' \
  -d '{"name":"Rohan","phone":"91XXXXXXXXXX","email":"you@example.com",
       "project":"XYZ Skyline Residences","source":"meta_ads","budget":"1.2 Cr"}'

# 2 · simulate each button without Meta wired up at all
curl -X POST http://localhost:5678/webhook/simulate \
  -H 'content-type: application/json' \
  -d '{"phone":"91XXXXXXXXXX","action":"VIEW_DETAILS"}'

# actions: VIEW_DETAILS · DOWNLOAD_BROCHURE · BOOK_VISIT
#          SLOT|<ISO datetime> · or {"text":"kya price hai?"}

# 3 · run the whole script
bash test/curl-examples.sh
```

> The `/simulate` endpoint fakes the *tap*, not the send — real WhatsApp messages still go out, so use your own number. While a workflow canvas is open in **Test** mode the paths are `/webhook-test/…`; once **Active** they are `/webhook/…`.

What to check after each test

- A row appeared in **Leads** with a grade and a score.
- The WhatsApp menu arrived with three buttons.
- **View Details** sent photos, then a summary with two buttons.
- **Download Brochure** delivered a real PDF, not the apology message.
- **Schedule Visit** offered three slots that are genuinely free on the calendar.
- Tapping a slot created the calendar event, invited the sales manager, and added a **Visits** row.

9 Connect the dashboard

1. Make sure **RE-06** is imported, its `SHEET_ID` and `DASHBOARD_KEY` are set, and it is **Active**.
2. Open `dashboard/index.html` in a browser — double-click it, or serve the folder:

```
cd dashboard && python3 -m http.server 8080   # then open http://localhost:8080
```

3. Click **Connection** and enter the webhook URL and the same key:

```
http://localhost:5678/webhook/dashboard-data
```

4. The pill in the top right turns green and shows the last refresh time. It re-reads every 30 seconds.

> Settings live in that browser's local storage only — nothing is sent anywhere, and each person who opens the file sets their own.

> **Do not publish this page on the public internet.** It reads your whole CRM through a key in a query string, which is fine on a laptop and not fine on a public URL. If you need remote access, put it behind your own auth or a VPN.

How the follow-up engine decides

The rules RE-04, RE-05 and the dashboard all share. Worth explaining to the client in these words.

Cadence by grade

GRADE	MEANING	CADENCE	6 FOLLOW-UPS SPANS
HOT	Stated budget that matches, visit interest, possession urgency, pre-approved loan	every 3 days	about 3 weeks
WARM	Genuine interest, vague on budget or timeline	every 5 days	about 1 month
COLD	Early browsing, out of budget, out of city, price curiosity	every 21 days	about 4 months
JUNK	Timepass: prank entries, gibberish, job seekers, vendor pitches, competitors, brokers	never	—

> **It is 6 touches, not 6 days**
>
> The gap between them depends on the grade, and the cap is a **safety stop, not a target**. Most leads book, reply, or get marked Lost long before touch 6. A HOT lead who has ignored six messages over three weeks is not hot any more — a human should take over, or the grade should drop.

Who is skipped, and why

SKIPPED	REASON
`Booked` `Lost` `Dropped`	Finished. Never touched again.
`Engaged`, conversation still live	A buyer who just replied is mid-conversation. A scheduled nudge landing on top of a live chat is the single most embarrassing failure this system can have. They resume automatically once quiet for `CONVERSATION_HOLD_HOURS` (default 72) — a permanent skip would drop everyone who ever replied out of nurture for good.
At the cap	6 automated follow-ups sent. Handed to a human.
No WhatsApp number (RE-05) / no email (RE-04)	Nothing to send to.

Three details that matter more than they look

followup_count, not touch_count

The cap counts *automated follow-ups only*. `touch_count` also rises on every button tap, so capping on it would lock an engaged buyer out of nurture after six taps. Replying resets the counter to zero — someone who re-engages earns a fresh sequence.

The write-back is the whole safety mechanism

`Update Touch Count` sets `last_touch` to now, so tomorrow's sweep skips the lead until the interval passes. If it failed silently the same buyer would be messaged **every single day** — worse than the workflow not running. So it retries three times and then emails the manager the exact row to fix.

Claude writes every message fresh

A fixed template is simpler, but follow-up 5 saying what follow-up 2 said is why nurture sequences get muted. Each message must carry one new reason to reply — a unit released, a possession update, a loan offer — escalating *value*, not pressure. "Just following up" is banned outright.

One predictable window

A lead due at 2pm is not messaged until the next morning. That is deliberate: all outbound lands in one window rather than pinging buyers at random hours, and a failed cron delays messages by a day instead of losing them.

These numbers are tunable and should be reviewed after a month of real data. Say that to the client — it signals you thought about their brand, not just the automation. Everything is in the `Config` node; change it in all three workflows together.

When something does not work

Ordered by how often it actually happens.

SYMPTOM	CAUSE AND FIX
Lead saved, no WhatsApp arrives	Almost always the 24-hour window. Either `USE_APPROVED_TEMPLATE` is `false` for a number that has not messaged you, or the template is not approved yet. Open the failed node's output — Meta's error names the reason.
Meta webhook verification fails	Workflow not Active, tunnel down, or the verify token does not match the n8n credential exactly.
Buttons arrive but taps do nothing	You did not subscribe to the `messages` field in Meta, or the tunnel URL changed since you saved the callback.
Photos or brochure never arrive	The URL in the Projects tab is not publicly fetchable. Drive share links return HTML. Test by opening the link in a private browser window — you should get the file itself.
Nothing is written to Sheets	`SHEET_ID` wrong, sheet not shared with the n8n Google account as Editor, or a column was renamed.
Everything is graded WARM with a flag	`ai_grading_unavailable` means the Claude call failed — bad key, no credit, or no network. The lead is deliberately still saved.
A lead is messaged twice in a morning	The cadence numbers in RE-04 and RE-05 have drifted apart, or RE-04's cron is no longer after RE-05's. Both must match.
A lead is messaged every day	The write-back is failing. Check the manager's inbox for the CRM write-back alert; it names the row and the fix.
`$vars` is empty	`$vars` is an Enterprise feature. These workflows deliberately use a `Config` node instead — if you added a `$vars` expression yourself, move it into Config.
Dashboard says "Cannot reach n8n"	RE-06 not Active, wrong URL, or n8n stopped. An inactive workflow only answers on `/webhook-test/`, and only while its canvas is open.
Dashboard says "n8n rejected the key"	Good news — the connection works. The key just does not match `DASHBOARD_KEY` in RE-06's Config.
Slots offered are already booked	`CALENDAR_ID` points at a different calendar from the one holding the events.

Where to look first, always

n8n's **Executions** list. Every run is there with the exact input and output of every node. A red execution tells you which node failed and why in one click — faster than any guess in this table.

Go-live checklist

Everything below should be true before the first real ad spend.

- [ ] Both WhatsApp templates show **Approved** in Meta.
- [ ] `USE_APPROVED_TEMPLATE` is `true` in RE-00.
- [ ] All four workflows are **Active**.
- [ ] The tunnel is on a stable domain, and `WEBHOOK_URL` matches it.
- [ ] Meta's callback URL is verified and subscribed to `messages`.
- [ ] Every project in the Projects tab has a price, photos and a public brochure link.
- [ ] The dashboard's Automation health page shows no warnings.
- [ ] A test lead on your own number completed all three buttons end to end.
- [ ] A test site visit appeared on the sales manager's calendar and in their inbox.
- [ ] Cadence numbers are identical in RE-04, RE-05 and RE-06.
- [ ] The sales manager knows what the at-the-cap list is and checks it.
- [ ] Someone owns the Projects tab and keeps it current.

> **After the first month**
>
> Review three things with real data: the cadence intervals, the 6-touch cap, and how Claude is grading. The dashboard's source table shows which channels actually convert — that is usually the first budget decision the client makes off the back of this system.